RELATÓRIO TÉCNICO / VERSÃO 0.4

# MicroLearn

## Sprint 4 — Consolidação, testes e preparação da entrega

Projeto de um Ambiente Virtual de Aprendizagem utilizando microsserviços e componentes de software reutilizáveis.

| Disciplina | Escopo | Situação |
|---|---|---|
| Engenharia de Software — 2026.2 | Projeto AVA em Python e React | Backend testado; validação do frontend e do Docker pendente |

Objetivo do documento. Apresentar o incremento da Sprint 4, consolidar a arquitetura e os contratos implementados, registrar as evidências de teste e especificar o que falta para entregar o trabalho acadêmico com capturas reais e repositório GitHub.

Aviso de validação. A conclusão dos testes Python não implica que a aplicação tenha sido compilada em React ou executada por Docker Compose. As verificações não executadas estão identificadas ao longo deste documento.

# 1. Escopo e metodologia de desenvolvimento

O sistema MicroLearn atende ao tema AVA definido na proposta da disciplina: modelagem e implementação de um sistema usando microsserviços e componentes de software. Agentes de IA são opcionais e não foram utilizados no protótipo. O recomendador é baseado em regras simples.

O planejamento utiliza Scrum com Product Backlog e quatro Sprints. O trabalho foi desenvolvido incrementalmente: primeiro a infraestrutura e a autenticação, depois cursos e matrículas, avaliações e recomendações e, nesta etapa, a finalização técnica e documental.

| Sprint | Escopo principal | Evidência |
|---|---|---|
| 1 | Gateway, Auth, JWT, frontend inicial | Login/cadastro e testes backend |
| 2 | Cursos, aulas, publicação, matrículas | Integração HTTP por perfis |
| 3 | Questionários, nota 0–10, recomendações | Teste da nota 5,0 e revisão |
| 4 | Gestão administrativa no frontend, verificação, documentação | 24 testes, status dos serviços, documentação e checklist |

## 1.1. Técnicas de reuso aplicadas

- Microsserviços: responsabilidades separadas por domínio, deploy e bancos independentes.
- Componentes React: Navbar, AppShell, Alert, CourseCard, CourseForm, LessonEditor, AssessmentEditor e ProtectedRoute.
- Bibliotecas e módulos Python: segurança JWT e validação de identidade reutilizadas entre serviços.
- Camada compartilhada de comunicação: apiClient e AuthContext reduzem duplicação de requisições e estado de sessão.

## 1.2. Resultado do incremento

A Sprint 4 adicionou uma tela de administração (`/admin/usuarios`) que permite listar usuários, mudar perfis e ativar ou desativar contas. Também foi incluída a atualização manual do estado dos microsserviços na Home, além da documentação de endpoints, validação e checklist de entrega.

# 2. Arquitetura geral do sistema

O navegador acessa o frontend React. As requisições de dados passam pela API Gateway em /api/v1. Os quatro serviços de negócio têm seus próprios processos e bancos SQLite. O gateway verifica JWT em rotas privadas, enquanto os serviços consultam a identidade atual no Auth Service e aplicam regras de autorização.

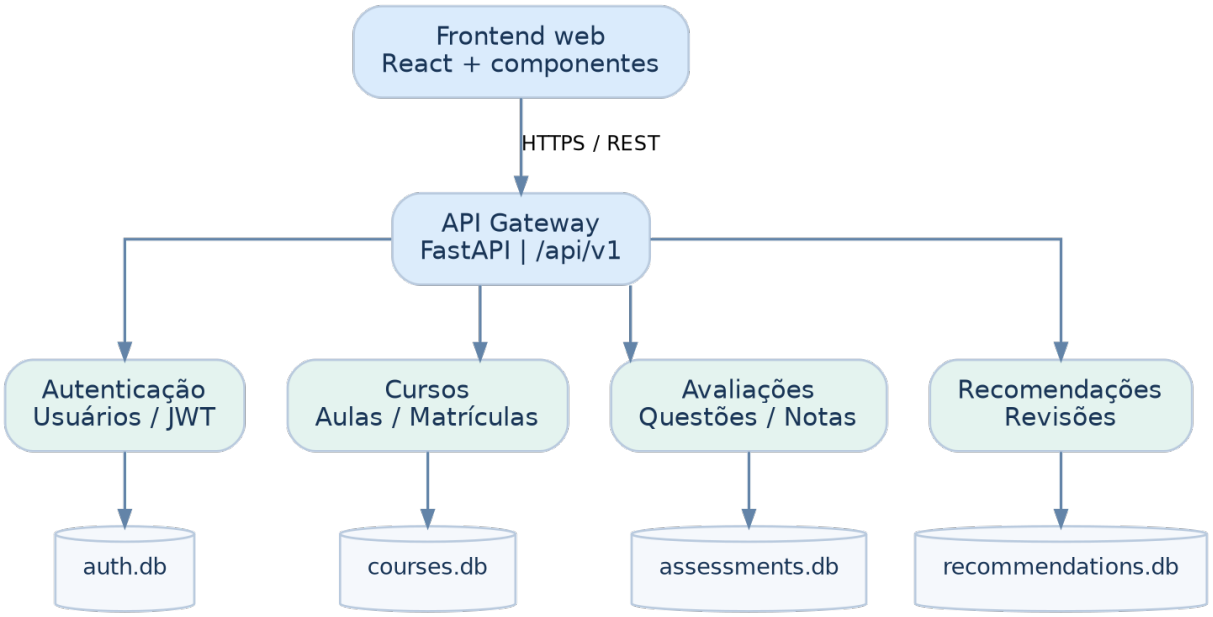



Figura 1 — Diagrama arquitetural elaborado na modelagem do MicroLearn. Os serviços executam de forma independente; as consultas entre domínios ocorrem por HTTP e não por acesso direto a bancos externos.

| Componente | Tecnologia | Responsabilidade |
|---|---|---|
| Frontend | React + Vite | Telas e componentes de interface |
| Gateway | FastAPI + httpx | Entrada HTTP, JWT, roteamento e status |
| Auth | FastAPI + SQLite | Usuários, login e papéis |
| Cursos | FastAPI + SQLite | Cursos, aulas e matrículas |
| Avaliações | FastAPI + SQLite | Questionários, tentativas e notas |
| Recomendações | FastAPI + SQLite | Regra nota < 7,0 e aulas de revisão |
| Orquestração | Docker Compose | Imagens, rede e volumes por serviço |

# 3. Contrato da API Gateway

O arquivo backend/gateway/main.py mantém o mapeamento dos serviços, encaminha o método/caminho/corpo da requisição e devolve a resposta HTTP. Um token JWT é obrigatório para rotas privadas; cadastro e login, além do catálogo de cursos, são públicos.

| Rota externa | Descrição / acesso |
|---|---|
| GET /api/v1/status | Retorna a situação atual da Gateway e dos quatro serviços; pública |
| POST /api/v1/auth/register | Cadastro de aluno; pública |
| POST /api/v1/auth/login | Autenticação; pública |
| GET /api/v1/auth/me | Perfil autenticado |
| GET /api/v1/auth/users | Lista de usuários; administrador |
| PATCH /api/v1/auth/users/{id} | Atualiza papel ou situação da conta; administrador |
| GET /api/v1/courses | Catálogo público de cursos publicados |
| POST /api/v1/courses | Criação de curso; professor |
| POST /api/v1/courses/{id}/enrollments | Matrícula; aluno |
| POST /api/v1/assessments/quizzes/{id}/submit | Respostas e nota; aluno |
| GET /api/v1/recommendations/mine | Recomendações do aluno |

A lista completa das rotas implementadas foi extraída dos decoradores do código FastAPI e está no arquivo docs/API_ENDPOINTS.md, dentro do ZIP.

## 3.1. Trecho de código documentado — consulta do status

`GET /api/v1/status` consulta os quatro endpoints `/health` de modo concorrente e devolve um objeto com `gateway`, `services` e `all_healthy`. Isso permite demonstrar os microsserviços independentes.

## 3.2. Possíveis respostas

200: sucesso; 201: recurso criado; 204: removido; 401: token ausente ou inválido; 403: acesso proibido; 404: recurso ausente; 409: conflito; 422: entrada inválida; 503: serviço indisponível.

# 4. Microsserviços e bancos de dados

| Serviço | Entidades locais | Operações principais |
|---|---|---|
| Autenticação | users | Registro, login, perfil, administração |
| Cursos | courses, lessons, enrollments | Curso, aula, matrícula |
| Avaliações | quizzes, questions, attempts | Quiz, gabarito privado, respostas, nota |
| Recomendações | recommendations | Sugestões recalculadas ao consultar |

## 4.1. Auth Service

O serviço cria contas públicas sempre como aluno, utiliza PBKDF2-SHA256 com salt para senhas e JWT para sessão. O administrador pode autorizar professores e desativar contas. A identidade é consultada novamente a cada operação relevante, impedindo que um JWT antigo preserve permissões revogadas.

## 4.2. Courses Service

O professor gerencia apenas os cursos de sua autoria, que começam como rascunho e podem ser publicados. Um aluno só consegue matricular-se em cursos publicados, e `UNIQUE(course_id, student_id)` bloqueia duplicidade. Aulas são acessadas por professor autor ou aluno matriculado.

## 4.3. Assessments Service

O professor cria questionários objetivos vinculados a cursos e aulas. O serviço nunca entrega o índice correto na consulta do questionário. As respostas são corrigidas no backend, com apenas uma tentativa por aluno e quiz.

## 4.4. Recommendations Service

Consulta os resultados no serviço Avaliações e as aulas disponíveis no serviço Cursos. Em vez de IA generativa, aplica regra explicável de desempenho, sem consultar diretamente bancos de outros domínios.

## 4.5. Decisões de persistência

Cada serviço utiliza um arquivo SQLite próprio e um volume independente no Compose. IDs UUID representam referências a entidades de outros serviços. A exclusão de cursos não é uma transação distribuída sobre questionários e notas: este é um limite conhecido da versão acadêmica.

# 5. Componentes de software reutilizáveis

A arquitetura do frontend utiliza componentes React para reaproveitar elementos de navegação, apresentação, formulários e autenticação. O desenho abaixo registra a organização proposta desses elementos.

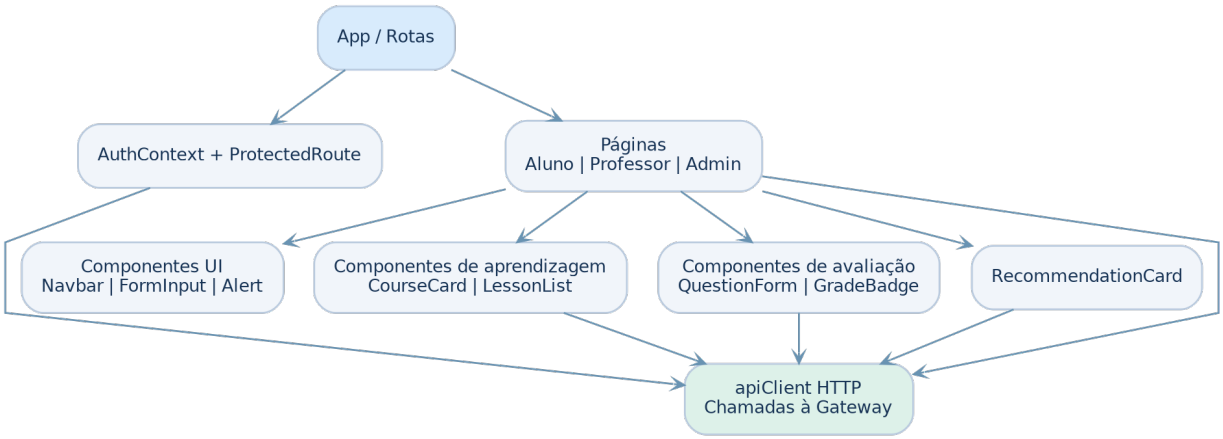



Figura 2 — Modelo dos componentes e de suas relações com as páginas do AVA.

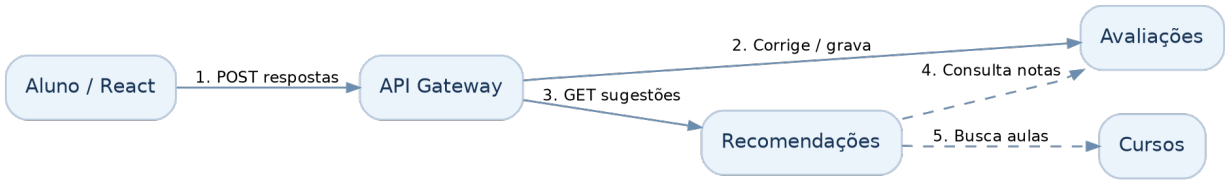
| Componente | Papel no reuso |
|---|---|
| AppShell e Navbar | Layout, menu e rodapé comuns a todas as páginas |
| ProtectedRoute | Barreira de sessão para rotas autenticadas |
| Alert | Avisos de erro/sucesso reutilizados em formulários e listas |
| CourseCard e CourseForm | Representação e criação de cursos |
| LessonEditor | Cadastro e edição de aulas |
| AssessmentEditor | Criação de questões com alternativas e gabarito |
| AuthContext | Token e perfil do usuário na interface |
| apiClient | Requisições REST centralizadas, incluindo admin |

## 5.1. Incremento visual da Sprint 4

A rota `/admin/usuarios` exibe uma tabela com nome, e-mail, perfil, situação, seletor de papel e botão de ativação/desativação. Alterações exigem confirmação e são transmitidas à API Auth. A Home fornece um botão de atualização do estado dos quatro serviços.

# 6. Regra de recomendação e fluxo funcional

A nota de um quiz é calculada por nota = arredondar(10 × acertos / total, 2). Quando nota < 7,0, o serviço sugere a aula de revisão que o professor associou ao questionário. Uma nota igual a 7,0 não gera recomendação.




| Etapa | Comportamento |
|---|---|
| 1. Professor | Cria curso, publica, adiciona aula e questionário |
| 2. Aluno | Inscreve-se, acessa aula e responde avaliação |
| 3. Avaliações | Confere matrícula, corrige respostas e registra nota |
| 4. Recomendações | Consulta notas e aula; sugere revisão quando nota é inferior a 7 |
| 5. Interface | Exibe nota e link do curso/aula de revisão |

## 6.1. Trecho conceitual fiel à implementação

```
score = round(10 * correct / len(rows), 2)
if result["score"] >= 7.0: continue
```

A primeira expressão pertence à correção do serviço Avaliações; a segunda é a condição que descarta notas sem necessidade de revisão no serviço Recomendações. Não há agentes de IA no código.

# 7. Telas e roteiro de demonstração

As interfaces abaixo estão presentes nos arquivos React. Este relatório não contém capturas de navegador, pois a instalação das dependências npm e o build não puderam ser executados neste ambiente. As capturas reais devem ser obtidas pelo grupo na máquina de desenvolvimento.

| Tela / URL | Demonstração esperada |
|---|---|
| / (Home) | Proposta do AVA e estado dos quatro serviços; botão Atualizar |
| /cadastro e /entrar | Novo aluno e autenticação |
| /admin/usuarios | Administrador lista contas e autoriza professores |
| /professor/cursos | Professor cria e gerencia cursos |
| /cursos e /cursos/:id | Catálogo, matrícula e aulas |
| /cursos/:id/avaliacoes | Professor cria quizzes; aluno consulta avaliações |
| /avaliacoes/:quizId | Aluno responde questões e consulta nota |
| /desempenho | Histórico de notas e aulas recomendadas |

## 7.1. Evidências exigidas na entrega

- Captura real de `docker compose ps` para mostrar Gateway, frontend e quatro microsserviços em execução.
- Captura real do painel da Home mostrando a consulta ao estado dos serviços.
- Capturas dos três perfis, do ciclo acadêmico e da interface de recomendações.
- Capturas ou trechos de código documentados para API Gateway, serviços e componentes.

O relatório definitivo para a disciplina deverá inserir essas imagens após execução local. Não devem ser substituídas por ilustrações fictícias.

# 8. Validação realizada e resultados

Os resultados registrados a seguir foram obtidos por execução local de Python com bancos temporários, e por verificação estática dos arquivos JavaScript/JSX.

| Verificação | Resultado real |
|---|---|
| pytest -q | 24 testes aprovados em aproximadamente 3 segundos |
| Smoke HTTP real | Gateway + quatro serviços em 5 processos independentes |
| Estado da Gateway | all_healthy = true; auth, courses, assessments, recommendations = ok |
| Exercício de notas | 1 acerto em 2 questões = 5,00 / 10 |
| Proteção de questionários | Consulta omite gabarito; segunda tentativa retorna 409 |
| Permissões | Aluno não matriculado bloqueado; professor rebaixado perde acesso |
| Administração | Promoção, desativação/reativação e autoproteção testadas |
| Sintaxe React | 24 arquivos JS/JSX analisados, 0 erros encontrados |
| Build React / Docker | Não executados; dependem do Windows e da rede |

## 8.1. Estado real dos microsserviços no teste HTTP

O script de integração obteve o seguinte resultado do endpoint `GET /api/v1/status`:

```
gateway: ok | all_healthy: true
auth: ok | courses: ok
assessments: ok | recommendations: ok
```

Esses valores são derivados da resposta HTTP real produzida durante o teste de cinco processos, arquivada em docs/smoke_test_results_sprint4.txt. Não representam uma execução no Docker.

## 8.2. Limites da validação

- O comando Docker não está disponível neste ambiente; o Compose não foi iniciado.
- A tentativa de `npm install` falhou por `EAI_AGAIN` no acesso ao registro npm; não houve build nem testes de navegador.
- A inspeção sintática do React não substitui a verificação das dependências, das rotas e da responsividade no navegador.

# 9. Execução reproduzível no Windows

Pré-requisitos: Python 3.11+ para testes locais; para execução completa, Docker Desktop e conexão para baixar imagens e dependências npm. Abra o PowerShell na pasta microlearn extraída do ZIP.

| Ação | Comando / instrução |
|---|---|
| 1. Preparar o ambiente | Copy-Item .env.example .env |
| 2. Criar segredo JWT | python -c "import secrets; print(secrets.token_urlsafe(48))" |
| 3. Editar .env | notepad .env — configurar JWT_SECRET, ADMIN_EMAIL, ADMIN_PASSWORD |
| 4. Construir serviços | docker compose up --build -d |
| 5. Conferir execução | docker compose ps |
| 6. Abrir aplicação | http://localhost:5173 |
| 7. Conferir Gateway | http://localhost:8000/api/v1/status |
| 8. Testar backend | python -m pytest -q ; python tests/smoke_processes.py |
| 9. Fazer build do React | cd frontend ; npm install ; npm run build |

## 9.1. Dicas de solução de problemas

- Se o Docker não estiver inicializado, iniciar Docker Desktop antes de executar o Compose.
- Se o frontend não abrir, examinar logs com `docker compose logs frontend` e `docker compose logs gateway`.
- Se a porta 5173 ou 8000 estiver em uso, encerrar o processo conflitante ou alterar o mapeamento no Compose.
- Não usar `docker compose down -v` sem intenção de apagar os bancos nos volumes.
- Não adicionar `.env` ao GitHub, pois contém segredos e credenciais.

# 10. Situação da entrega e pendências

| Item exigido no relatório da disciplina | Situação |
|---|---|
| Enunciado / técnicas de reuso / método ágil | Documentados nas etapas 1 a 4 |
| Product Backlog e Sprint Backlogs | Documentados na Etapa 4 |
| Arquitetura de microservices | Diagramas disponíveis em docs/ e etapas anteriores |
| Gateway e código documentado | Código + docs/API_ENDPOINTS.md |
| Especificação de cada serviço e componente | Arquivos independentes + seções deste relatório |
| Sistema de recomendação | Implementado por regras + documentado |
| Lista de microservices em execução | Status HTTP validado; captura real do Docker ainda pendente |
| Tecnologias por serviço e bancos | Documentado neste relatório |
| Telas explicadas passo a passo | Roteiro pronto; imagens reais ainda pendentes |
| Código funcional no GitHub e link na capa | ZIP preparado; publicação/URL pendentes |
| Referências bibliográficas | Relacionadas na seção seguinte |

## 10.1. Limitação arquitetural a registrar

A exclusão de um curso ou aula não inicia uma transação distribuída para remover questionários, tentativas e notas do banco de Avaliações. O recomendador descarta recursos sem acesso ou ausentes, mas o histórico das avaliações pode persistir. Em produção seria necessária uma política de exclusão, retenção e consistência entre serviços.

## 10.2. Próximo marco de trabalho

Validar o Docker e o build React no Windows, obter capturas de tela reais, verificar navegação responsiva, registrar integrantes e orientador e publicar o código-fonte no GitHub. Depois inserir a URL real na capa do relatório acadêmico e reunir as etapas de modelagem e implementação em um documento definitivo.

# 11. Referências e rastreabilidade

As referências de tecnologia e metodologia abaixo orientam os conceitos empregados. Os detalhes de comportamento descritos neste relatório foram conferidos diretamente no código-fonte do MicroLearn e nos registros locais de teste.


- FOWLER, Martin; LEWIS, James. Microservices. 2014. Disponível em: https://martinfowler.com/articles/microservices.html.
- SCHWABER, Ken; SUTHERLAND, Jeff. The Scrum Guide. 2020. Disponível em: https://scrumguides.org/scrum-guide.html.
- FASTAPI. FastAPI Documentation. Disponível em: https://fastapi.tiangolo.com/.
- META OPEN SOURCE. React Documentation. Disponível em: https://react.dev/.
- DOCKER. Docker Compose Documentation. Disponível em: https://docs.docker.com/compose/.
- SQLITE. SQLite Documentation. Disponível em: https://www.sqlite.org/docs.html.


## Arquivos que acompanham o projeto

| Arquivo | Conteúdo |
|---|---|
| README.md | Preparação, rotas, execução e limites conhecidos |
| docs/API_ENDPOINTS.md | Tabela de endpoints extraída dos serviços FastAPI |
| docs/VALIDACAO_SPRINT4.md | Resultados verificados e limitações do ambiente |
| docs/CHECKLIST_ENTREGA.md | Itens concluídos e pendências para envio acadêmico |
| docs/pytest_results_sprint4.txt | Saída real do pytest |
| docs/smoke_test_results_sprint4.txt | Saída do teste HTTP real, incluindo status dos serviços |
| docs/*.png e docs/arquitetura.mmd | Diagramas de arquitetura, componentes e fluxo de avaliação |

Nota editorial: complete com o nome dos integrantes e o link real do repositório GitHub antes da entrega acadêmica. Não foram inventadas imagens de interface nem resultados de Docker.